Facultatea de Automatică şi Calculatoare, UPB
Departamentul de Calculatoare

Analiza algoritmilor

ŞL.dr.ing. Andrei Mogoş

2020-2021

- Introducere -

Notare

- **Laborator: 6p**
  - 2 teme: 3p (2 x 1.5p)
  - teste de curs: 1.5p
  - activitate de laborator: 1.5p
- **Examen: 4p**

- Condiţie de intrare în examen:

  Laborator >= 3p
- Condiţie de promovare a examenului:

  Examen >= 2p

Capitolele cursului

- Complexitatea algoritmilor
- Corectitudinea algoritmilor
- Clase de probleme
- Algoritmi de aproximare
- Calculabilitate

Bibliografie

- C.A. Giumale, “Introducere in analiza algoritmilor”, Polirom, 2004
- T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein, “Introduction to algorithms”, Second Edition, MIT Press, 2001

Problemă → Algoritm → Program (1)

1. Specificarea problemei
2. Analiza problemei din punct de vedere al rezolvabilităţii mecanice
3. Stabilirea clasei din care face parte problema => tipul algoritmului (exact, de aproximare, euristic, …)
4. Alegerea paradigmei de programare (alegerea tipului de limbaj de programare)

Problemă → Algoritm → Program (2)

5. Dezvoltarea unui algoritm

   - depinde de clasa din care face parte problema

   - depinde de paradigma de programare

6. Analiza corectitudinii algoritmului

7. Analiza complexităţii algoritmului

8. Codificarea algoritmului într-un limbaj de programare corespunzător paradigmei de programare alese.

Problemă (1)

Definitie: Se numeşte **problemă** o mulţime de întrebări referitoare la proprietăţile dinamice sau structurale ale unor entităţi (procese sau obiecte), întrebări care acceptă răspunsuri precise, finite, a căror corectitudine poate fi riguros demonstrată.

Universul problemei = mulţimea entităţilor

Datele problemei = partea universului problemei considerată cunoscută

Soluţiile problemei = răspunsurile la întrebări

Problemă (2)

Considerăm două tipuri de probleme:

a) **calculul unei funcţii**: a rezolva problema înseamnă a calcula funcţia

Exemplu: Se dă x număr real. Care este modulul său?

b) **deciderea unei mulţimi**: a rezolva problema înseamnă a decide dacă o valoare aparţine unei mulţimi date

Exemplu: Se dă n număr natural. Este n număr prim?

Problemă (3)

Observaţie: Deciderea $x \in M$ este echivalentă cu calculul funcţiei $test_M$ în x. Funcţia $test_M$ are forma:

$$test_M : M \to \{0,1\}$$

$$test_M(x) = \begin{cases} 1, & x \in M \\ 0, & x \notin M \end{cases}$$

Calculabilitate (1)

Definiţie: **O procedură efectivă** este o mulţime finită de instrucţiuni, finite ca dimensiune a reprezentării, a căror execuţie se termină în timp finit şi implică o cantiate finită de date pentru a obţine un rezultat, mereu acelaşi pentru date identice.

Definiţie: O funcţie $f : I \to O$ se numeşte **efectiv calculabilă** dacă există o procedură efectivă capabilă să calculeze valoarea funcţiei pentru fiecare punct $x \in I$.

Calculabilitate (2)

Definiţie: **Un model teoretic al calculabilităţii** este un formalism conform căruia se execută o procedură efectivă.

Modele teoretice ale calculabilităţii:

- Maşina Turing (Turing, 1936)
- Funcţiile recursive (Kleene, 1936)
- Calculul Lambda (Church, 1936)
- Algoritmii normali (Markov, 1954)

Calculabilitate (3)

Observaţie: Modelele teoretice prezentate pe slide-ul anterior sunt echivalente din punct de vedere computaţional: orice calcul efectuat într-un model poate fi efectuat în toate celelalte modele.

Teza Church-Turing

O funcţie este efectiv calculabilă dacă şi numai dacă este Turing calculabilă.

Observaţie: O teză este o propoziţie prespusă a fi adevarată, dar care nu a putut fi demonstrată ca fiind adevarată (teza afirmă, dar nu demonstrează).

Calculabilitate (4)

Definiţie: **O maşină de calcul** este o implementare a unui model teoretic al calculabilităţii. Se consideră că o maşină de calcul are resurse de calcul finite, dar oricat de mari.

Observaţie: O procedură efectivă nu se adresează unui model teoretic al calculabilităţii şi deci nu se adresează unei anumite maşini de calcul. Conceptul este general şi se referă la capacitatea de a executa mecanic, indiferent de maşina de calcul, o mulţime de instrucţiuni în condiţiile unor restricţii aplicate datelor prelucrate şi instrucţiunilor.

Calculabilitate (5)

Observaţie: Calculabilitatea efectivă a unei funcţii se referă la existenţa unei masini de calcul şi a unui program care prin execuţie pe maşina de calcul să calculeze valoarea funcţiei în orice punct din domeniul de definiţie.

Algoritm (1)

Definiţie: Se numeşte **algoritm** specificarea unei proceduri efective folosind o notaţie convenţională, conform particularităţilor computaţionale ale unei maşini de calcul.

Definiţie: **Un algoritm acceptat de o maşină de calcul** este o mulţime finită de instrucţiuni, care pot fi executate de către maşina de calcul în timp finit pentru a calcula soluţia unei probleme.

Observaţie:

Funcţie = specificarea unei probleme

Algoritm = mod de rezolvare a problemei

Algoritm (2)

Observaţie: **O funcţie** $f : I \to O$ este o mulţime de perechi $\{(x, f(x)) \mid x \in I,\ f(x) \in O\}$ ce asociază un rezultat fiecărei valori a argumentului funcţiei, **dar nu arată modul în care rezultatul poate fi calculat**.

Observaţie: **Un algoritm** este un şir de simboluri (un text) care indică modul şi ordinea în care trebuie transformate datele de intrare pentru a obţine rezultatul corespunzător.

Complexitatea unui algoritm

Teoria complexităţii se ocupă de problemele rezolvabile mecanic, subiectul principal fiind studiul performanţelor algoritmilor de rezolvare pentru aceste probleme.

Performanţele unui algoritm se referă la cantitatea de resurse de calcul (timp şi spaţiu de memorie) necesare execuţiei algoritmului.

Complexitatea temporală a unui algoritm se referă la timpul de execuţie al algoritmului.

Complexitatea spaţială a unui algoritm se referă la spaţiul de memorie necesar execuţiei algoritmului

Corectitudinea unui algoritm (1)

Analiza corectitudinii unui algoritm se referă la căutarea răspunsului la întrebarea:

> "**rezultatele calculate de algoritm corespund soluţiilor problemei?**"

Se studiază două aspecte:

- terminarea algoritmului
- validarea logică a algoritmului

Corectitudinea unui algoritm (2)

Observaţie: **Terminarea algoritmilor** este o problemă nerezolvabilă mecanic. Nu există tehnici generale de demonstrare a terminării unui algoritm; fiecare caz trebuie tratat separat.

Observaţie: Pentru **validarea logică a algoritmului** se demonstrează că rezultatul calculat de algoritm corespunde soluţiei problemei. Există tehnici generale de demonstrare, dar aplicarea lor este dependentă de problemă.

Limbaj de programare (1)

Definiţie: **O paradigmă de programare** este o mulţime de concepte de programare şi de construcţii de programare induse de un anumit model teoretic al calculabilităţii.

Definiţie: **Un limbaj de programare** este o modalitate de a exprima algoritmii pentru o anumită paradigmă de programare.

Observaţie: Unei paradigme de programare ii corespund unul sau mai multe limbaje de programare.

Limbaj de programare (2)

Limbaje de programare:

1. **declarative** (Ce acţiuni trebuie făcute pentru a rezolva problema?)
   - **funcţionale**
   - **logice**
   - **asociative**

2. **imperative** (Cum trebuie să se execute o mulţime de acţiuni pentru a rezolva problema?)
   - **von Neumann** (limbajele de programare convenţionale)

Limbaj de programare (3)

Limbaje funcţionale: LISP, Racket (Scheme), Haskell
Limbaje logice: Prolog
Limbaje asociative: CLIPS
Limbaje imperative: C/C++, Java, C#

Observaţie:
Limbaje declarative – paradigma declarativă
  Limbaje funcţionale – (sub)paradigma funcţională
  Limbaje logice – (sub)paradigma logică
  Limbaje asociative – (sub)paradigma asociativă
Limbaje imperative – paradigma imperativă